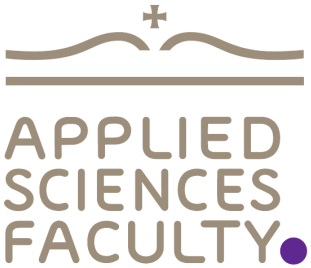
C++ — лекція 14. Успадкування





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Розглянемо на прикладі автотранспорту.

Будь-який автотранспорт вміє:

- Рушати.
- Збільшувати швидкість.
- Гальмувати.
- Повертати.
- Перевіряти свій стан.
- І т.д.

Автобус є видом автотранспорту. Для нього актуально все, що стосується автотранспорту.
Однак, він також вміє:

- Здійснювати посадку-висадку пасажирів.

Вантажівка теж є видом транспорту. Для неї є важливими наступні операції:

- Завантаження.
- Розвантаження.

Екскаватор ж:

- Повинен виконувати найрізноманітніші маніпуляції ковшем.

При чому, операції з ковшем не мають сенсу для автобуса, а посадка пасажирів — для екскаватора.
Крім того, неможливо, та й не потрібно, передбачити наперед всі можливі потреби.

Для вирішення цієї проблеми існує другий найважливіший засіб ООП — **успадкування**.

Успадкування — це коли з певного класу, базового класу, (предка або суперкласу), породжується його нащадок — підклас, породжений клас, який **вміє все те**, що і його предок, а також ще щось.

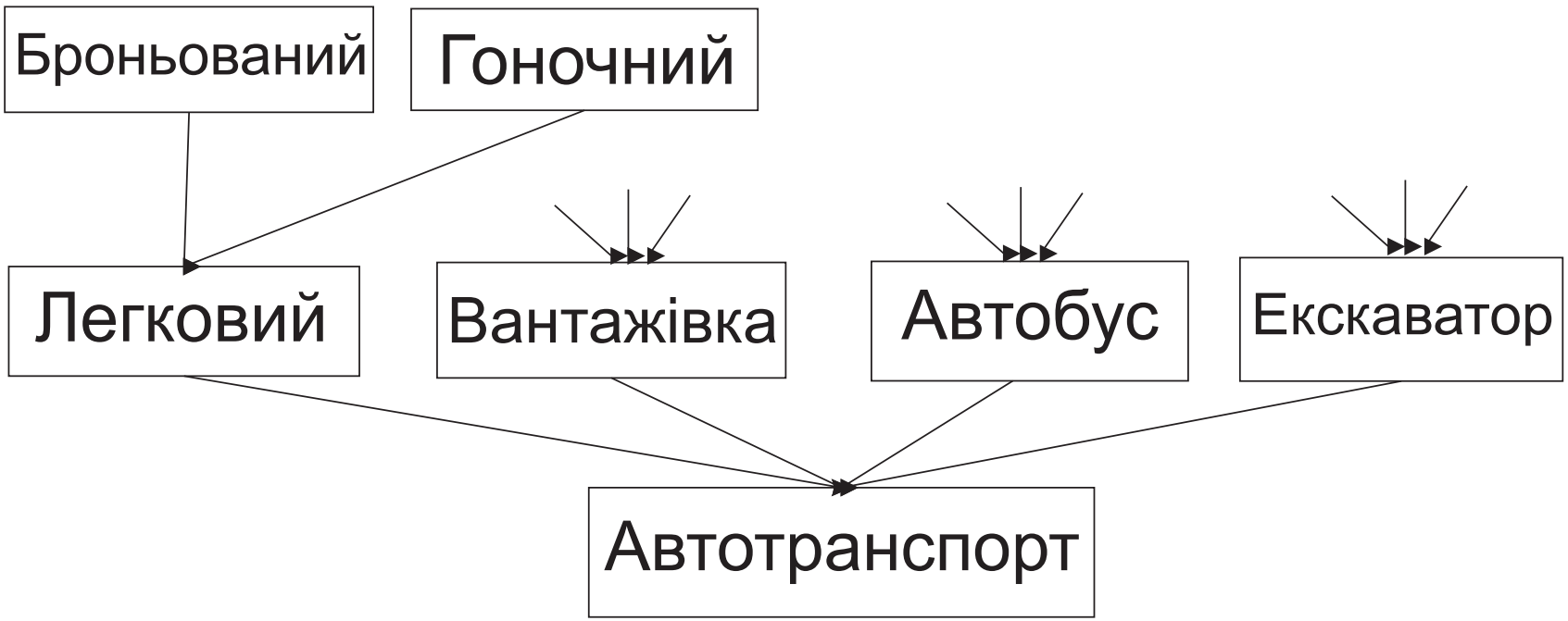
Наприклад: автотранспорт — базовий клас. Від нього успадковують автобус, вантажівку, легковий автомобіль, екскаватор. Виникне потреба описати і бульдозер — треба буде всього лиш створити ще один клас-нащадок від автотранспорту.

Успадкування — засіб використання коду, що вже існує, щоб не повторювати одну і ту ж роботу. Також, це засіб розширення функціональності.

УВАГА! Якщо клас-нащадок успадковується від базового класу це означає тільки одне: що цей клас Є ТИМ ЖЕ, ЩО І ПРЕДОК плюс ЩЕ ЩОСЬ!

Тобто, твердження ”нащадок є предком” завжди істинне.

БЕЗ ЖОДНИХ ВИНЯТКІВ!


Броньований
Гоночний
Легковий
Вантажівка
Автобус
Екскаватор
Автотранспорт

Публічне успадкування

Публічне успадкування в C++ відповідає описаному вище "класичному" успадкуванню — клас розширяє свого предка. Його стосується твердження "клас є різновидністю предка".

```
class Base{
private:
    int private_method();
protected:
    void protected_method();
public:
    double public_method();
};
class Derived : public Base {
public:
    void some_fn();
};
```

При цьому класу Derived будуть доступні методи protected_method та public_method(), так само, як оголошені безпосередньо в ньому методи. Метод private_method доступним не буде!

- Нащадок успадковує всі поля та методи предка, крім: конструкторів, деструкторів, операторів присвоєння та друзів.
- Хоча приватні поля успадковуються, клас-нащадок не має до них доступу і може з ними працювати лише за допомогою відкритого або захищеного інтерфейсу предка.
- Якщо клас-нащадок не додає нові поля, його розмір збігатиметься із розміром предка.
- Клас-нащадок може мати поля із тим же іменем, що й поля предка. Вони приховують відповідні поля базового класу, і для доступу до них слід використовувати кваліфікатор імені базового класу, наприклад **Base::fld**.
  - ◇ Не варто цей засіб використовувати навмисне, але він допомагає з зворотною та майбутньою сумісностями, див., наприклад, обговорення в "Is shadowing a member variable from a base class a bad thing? Maybe, but maybe not."
- Клас-нащадок може мати методи із тими ж іменами, що й предок. Так робити категорично не варто! Вони перевизначають відповідні методи предка.
  - ◇ **Увага!**, невидимими робляться всі методи предка із цим іменем. Для звертання до них можна використати кваліфікатор доступу.
- Статичні методи та члени теж успадковуються. Однак, всі нащадки ділять одну їх копію! Звертатися до них можна як за допомогою кваліфікатора базового класу, так і кваліфікаторів нащадків.

Невірний код, лише для ілюстрації

- Якщо нащадок означає не віртуальну функцію із тим же іменем, що були у базовому класі, вона приховує всі функції із тим іменем з базового класу.
- Явно її можна викликати, вказавши повне ім'я із кваліфікатором доступу.

```
class Base{
public:
    int method1(){ return 1;}
    int method1(int x) {return 2*x;}
};

class Derived: public Base {
public:
    int method1(){return 17;};
};
 ........
        Derived d;
        d.method1();
        //d.method1(33); <=== Помилка!
        d.Base::method1(33);
```

- Щоб перевизначені функції брали участь в перевантаженні методів класу-нащадка, можна використати using-оголошення. Воно "притягує" всі відповідні методи предка, але при збігу сигнатур вищий пріоритет матимуть методи нащадка.

```
class Derived: public Base
{
public:
    using method1;
    int method1(){return 17;}
};
  ........
        Derived d;
        d.method1();          // Derived::method1
        d.method1(33);        // Тепер компілюється і це
        d.Base::method1(33);
```

- using, використане в приватній частині класу, зробить відповідні функції приватними, навіть якщо вони були відкритими у класі-предку.
- Таке використання using можливе завдяки тому, що клас вважається спеціальним простором імен.

Конструктори не успадковуються (до C++11)

- Якщо в базовому класі є конструктор за замовчуванням, конструктор базового класу можна не писати — він буде створюватися автоматично.
- Якщо конструкторів за замовчуванням немає, клас-нащадок повинен мати конструктор, що викликає конструктор базового класу:

```
class Base2{
public:
    Base2(int x){}
};

class Derived2: public Base2{
public:
    Derived2(int x, double y):
      Base2(x)
    {}
};
```

Конструювання відбувається знизу вверх — спочатку конструюється "найбільш базовий" клас, а далі, послідовно, всі його нащадки. При тому, кожен із них має можливість налаштувати свої члени даних згідно своїх потреб.

Успадкування конструкторів — C++11

- C++11 додає можливість успадковувати конструктори базових класів, за допомогою відповідного using-оголошення:

```
class CB{
public:
    CB(int x){}
};

class CD: public CB{
public:
    using CB::CB;
};
    ..................
    CD d(3);
```

- При тому, ініціалізація полів-об'єктів все рівно відбуватиметься – викликом конструкторів за замовчуванням.
- ◇ Поля, які вимагають явної ініціалізації, (фундаментальні типи та агрегати, що їх включають), залишаться неініціалізованими[1].

[1] Формально в стандарті це формулюється по іншому, але наслідок саме такий.

Деструктори не успадковуються

- Деструктор базового класу викликається при знищенні предка автоматично.
- Деструктори викликаються у зворотному порядку до описаного вище порядку виклику конструкторів.
- Однак, якщо деструктор базового класу був віртуальним, деструктор нащадка, навіть автоматично створений, теж буде віртуальним.

Оператори присвоювання теж слід реалізовувати у кожному нащадку

- Зрозуміло, що оператор присвоювання нащадка повинен копіювати і частину, що прийшла з предка, за допомогою його відповідного оператора:

```
class Base {
    .....
public:
    Base(const Base&) { .... }
    Base& operator=(const Base&) { ..... }
};
class Derived : public Base {
    int newfield;
public:
    Derived(const Derived& rhs):
        Base(d) // Викликаємо конструктор копіювання предка
        , newfield(rhs.newfield)
    { ..... }
    Derived& operator=(const Derived& rhs) {
        Base::operator=(rhs);
        newfield = rhs.newfield;
        return *this;
    }
};
```

- Якщо влаштовує предковий оператор, в C++11 можна написати:

```
using Base::operator=;
```

Закрите успадкування

- Закрите успадкування відповідає успадкуванню реалізації без успадкування інтерфейсу.
- Тобто, клас, який успадковує закрито, не є розширенням базового класу! Він просто успадковує реалізацію, але реалізує за її допомогою інший інтерфейс.
- Синтаксис:

```
class Base {    .....    };

class SomeClass : private Base {    .....    };
```

При цьому, захищені та публічні члени даних та методи стають закритими членами класу-нащадка.

Закрите успадкування

- За потреби, клас може надати своїм користувачам доступ до частини успадкованих приватно методів.
- Надаємо доступ до fn1() і fn2(), але НЕ fn3():

```
class Base {
public:
    void fn1(){};
    void fn2(){};
    void fn3(){};
};
class SomeClass : private Base {
public:
    using Base::fn1;
    using Base::fn2;
};

    .....
    SomeClass cc;
    cc.fn1();          // OK
    // cc.fn3();       // Error!
```

- Доступ можна надати лише для того, до чого має доступ сам клас — зокрема, він не може надати доступу до приватних членів базових класів.

Приклад закритого успадкування

Універсальний типо-безпечний клас стеку можна оголосити так:

```
template <class SomeType>
class OutStack{
    SomeType *data;
    size_t stack_size, top;
public:
    OutStack(size_t st_size):
      stack_size(st_size),top(0) {
        data = new SomeType[stack_size];
    }
    ~OutStack(){
        delete [] data;
    }
    void push(const SomeType& in);
    SomeType pop();
    bool is_empty() const;
};
```

Однак, для кожного нового типу відбуватиметься генерація окремого коду — "розбухання" шаблонів. При чому, код буде практично ідентичним.

Якщо достатньо, щоб стек міг працювати лише із POD-типами, можна зробити так:

```
class OutVoidStack{
private:
    void *data;
    size_t stack_size, element_size, top;
public:
    OutVoidStack(size_t st_size, size_t elem_size):
      stack_size(st_size),element_size(elem_size), top(0){
        data = ::operator new (st_size*elem_size);
    }
    ~OutVoidStack(){
        // Code to call objects destructors
        ::operator delete (data);
    }

    void push(const void* in);
    void* pop();
    bool is_empty() const;
};
```

Код не дублюватиметься, але працювати із таким класом більш ніж незручно — він занадто небезпечний!

Варіант — забороняємо створювати екземпляри OutVoidStack, і створюємо від нього типобезпечні нащадки.

```
class OutVoidStack{
private:
    .....
protected:
    OutVoidStack(int st_size, int elem_size):
      stack_size(st_size),element_size(elem_size), top(0){
        data = malloc(st_size*elem_size);
    }
    ~OutVoidStack(){
        free(data);
    }

    void push(const void* in);
    void* pop();
    bool is_empty() const;
};
```

```
template <class T>
class OurStack: private OutVoidStack
{
public:
        typedef T value_type;
    OurStack(size_t st_size):
      OutVoidStack(st_size, sizeof(value_type) )
    {}

        inline void push(const T* in){OutVoidStack::push(in);}
        inline T* pop(){ return static_cast<T*>( OutVoidStack::pop() ); }
        inline bool is_empty() const { return OurVoidStack::empty(); }

};
```

Ці класи практично не генеруватимуть свого реального коду, при тому, значно безпечніші та зручніші від свого предка.

З використанням віртуальних функцій можна таким чином зробити і довільним стеком, не тільки таким, що обмежується використанням POD. Див., наприклад, `https://eugenedruy.wordpress.com/2009/07/19/refactoring-template-bloat/`.

Захищене успадкування

- Використовується рідко.
- Ідейно близьке до приватного, тільки приховування реалізації відбувається лише від користувачів, не від нащадків.
- Зокрема, захищені та публічні члени класу-предка стають захищеними членами нащадка.
- Одне із можливих використань — коли є потреба реалізувати якийсь інтерфейс (успадкувавши його від базового класу), використовуючи для реалізації інший, але так, щоб він не був доступний користувачам. Тоді приватне успадкування буде надміру обмежувати — нащадки не зможуть перевантажувати віртуальні функції класу-реалізації, природним буде зробити так:

```
class PushButton: public AbstractButton, protected Dos64ApiGuiButton
  {.....};
```

Детальніше див., наприклад, в книзі Страуструпа, "The C++ Programming Language " глава 21[2] "Ієрархії класів".

[2] У виданні 2013 року (C++11), в старому — виданні — глава 16.

Принцип підстановки

- Оскільки об'єкт-нащадок[3] є предком, його можна використовувати всюди, де може використовуватися предок. Це – так званий принцип підстановки Лісков.
- ◇ Зворотне, зрозуміло, не буде істинним!
- Всюди, де використовуються вказівники чи посилання на об'єкт-предок, можна передавати і його нащадків.
- В цілому, оскільки різні нащадки мають різний розмір, зручніше працювати з вказівниками чи посиланнями на об'єкти.
- ◇ Один із винятків – боротьба за продуктивність. Для сучасних мікроархітектур може бути корисним, замість масиву вказівників на поліморфні[4] об'єкти скористатися масивом std::variant, структурою масивів – по масиву на тип тощо.
- ◇ Див. приклади: "Process polymorphic classes in lightning speed"

[3] За публічного успадкування.

[4] Див. далі.

Зрізка

- В принципу підстановки є побічний ефект.
- Типові оператори присвоєння виглядають так:

```
Base& Base::operator=(const Base& rhs);
Derived& Derived::operator=(const Derived& rhs);
```

- Завдяки принципу підстановки, крім присвоєнь:

```
Base b1, b2;
Derived d1, d2;
b1 = b2;
d1 = d2;
```

- дозволеним буде і таке:

```
b1 = d2;
```

- При цьому, однак, відбувається **зрізка** — в b1, потрапляє лише та частина d1, що успадкована від Base.
- Це одна з причин, чому рідко використовуються контейнери об'єктів для класів з використанням успадкування.

Посилання

- "Techniques for post-facto multiple inheritance" – деякі трюки як отримати множинне успадкування для незалежних функцій.

